Dictionaries and Mini Project

Welcome to an exciting journey into Python dictionaries! In this session, you'll master one of Python's most powerful data structures and apply your knowledge to build real-world programs. Dictionaries are everywhere in programming, from storing user data to managing configuration settings. By the end of this lesson, you'll have the skills to create interactive programs that store and retrieve information efficiently.

What is a Dictionary?

Key-Value Pairs

A dictionary is like a real-world dictionary where you look up a word (the key) to find its definition (the value). In programming, dictionaries store data as key-value pairs, making it easy to organize and retrieve information.

Why Use Dictionaries?

- Fast data retrieval using keys
- Flexible data organization
- Store related information together
- Dynamic size - add or remove data easily

Creating Your First Dictionary

Dictionary Syntax

Use curly braces `{}` and separate key-value pairs with colons

Keys

Can be strings, numbers, or other immutable types. They must be unique.

Values

Can be any data type - strings, numbers, lists, or even other dictionaries!

```
student = {
    "name": "Arun",
    "age": 20,
    "grade": "A",
    "courses": ["Math", "Science", "English"]
}
```

This dictionary stores all information about a student in one organized structure. Notice how different data types (strings, integers, lists) can coexist in the same dictionary!

Accessing Dictionary Values

Retrieving Data

Access values using their corresponding keys inside square brackets `[]`. This is much faster and more intuitive than using indexes like in lists.

```
student = {
    "name": "Arun",
    "age": 20,
    "grade": "A"
}

print(student["name"])  # Output: Arun
print(student["age"])   # Output: 20
```

Common Mistake

Trying to access a key that doesn't exist will cause an error. Always ensure the key exists or use error handling techniques.

Adding and Modifying Data

01

Add New Key-Value Pair

Use the key in square brackets and assign a value

02

Update Existing Value

Assign a new value to an existing key

03

Check the Result

Print or examine the modified dictionary

```
student = {"name": "Arun", "age": 20}

# Adding a new key
student["marks"] = 85
print(student)
# Output: {'name': 'Arun', 'age': 20, 'marks': 85}

# Updating an existing value
student["age"] = 21
print(student)
# Output: {'name': 'Arun', 'age': 21, 'marks': 85}
```

Looping Through a Dictionary

Dictionaries are iterable, meaning you can loop through them to process all key-value pairs. The `.items()` method returns both keys and values, making it perfect for displaying or processing dictionary contents.

```
student = {"name": "Arun", "age": 20, "marks": 85}

for key, value in student.items():
    print(f"{key}: {value}")

# Output:
# name: Arun
# age: 20
# marks: 85
```

Other Useful Methods

`.keys()` returns all keys, `.values()` returns all values, and `.items()` returns both as pairs.

Getting User Input for Dictionaries

Dynamic Data Entry

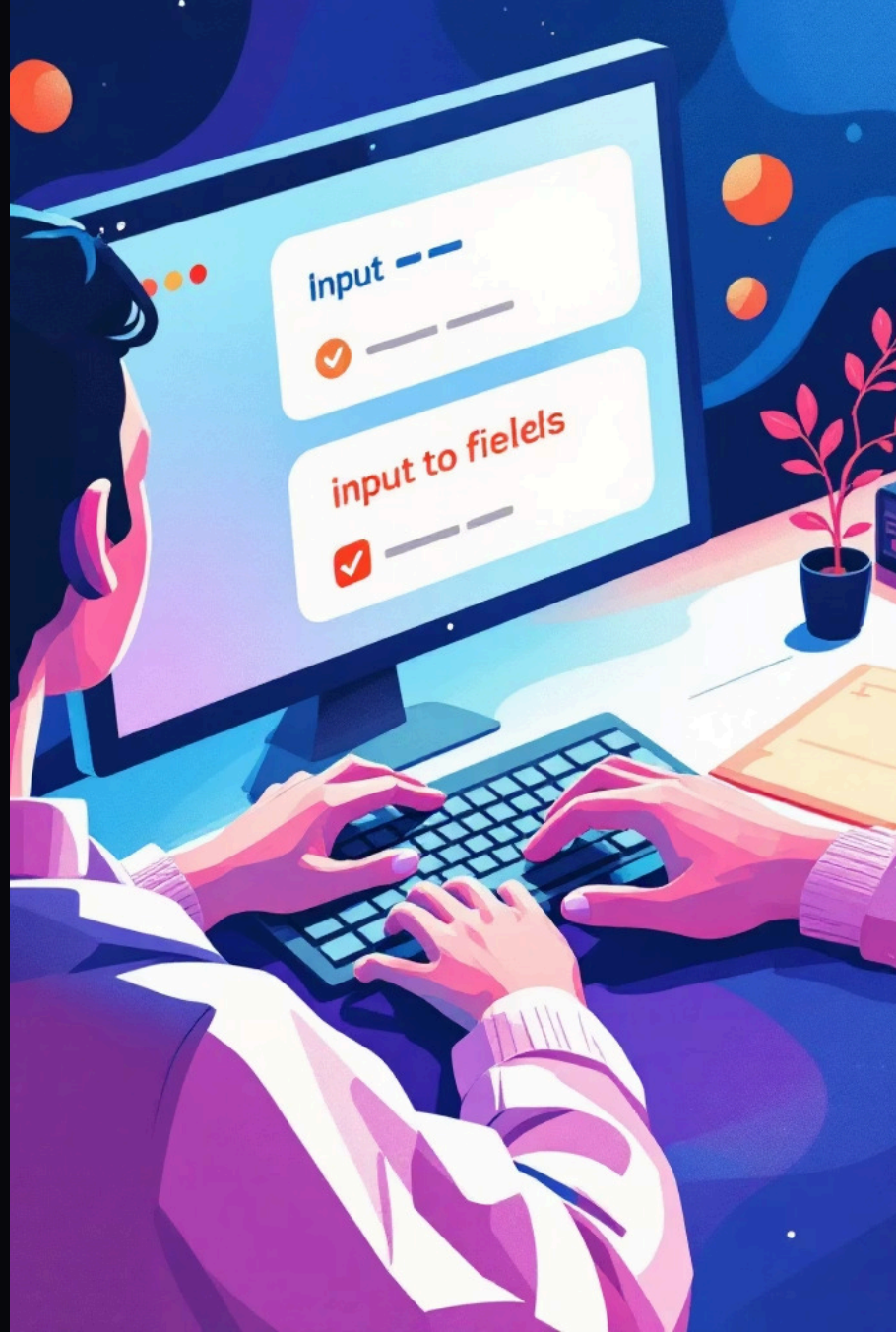
Combine the `input()` function with dictionaries to create interactive programs. This allows users to provide data that gets stored directly into your dictionary structure.

```
name = input("Enter name: ")
age = input("Enter age: ")
marks = input("Enter marks: ")

student = {
    "name": name,
    "age": int(age),
    "marks": int(marks)
}

print(student)
```

The `int()` function converts string input to integers. This is important because `input()` always returns strings, and you might need other data types for calculations or comparisons.



Storing Multiple Students

Create an Empty List

Start with a list to hold all student dictionaries

Build Student Dictionary

Create a dictionary with student data

Add to List

Use `append()` to add the dictionary to the list

```
students = []

# Add first student
students.append({
 "name": "Arun",
 "age": 20,
 "marks": 85
})

# Add second student
students.append({
 "name": "Priya",
 "age": 19,
 "marks": 92
})

print(students)
```

Mini Project: Student Management Menu

Now it's time to put everything together! Build a complete text-based menu system that manages student records. This project combines dictionaries, lists, loops, and user input to create a functional program.

```
students = []

while True:
 print("\n1. Add Student")
 print("2. View Students")
 print("3. Exit")

 choice = input("Enter your choice: ")

 if choice == "1":
 name = input("Enter name: ")
 age = input("Enter age: ")
 students.append({
 "name": name,
 "age": age
 })
 print("Student added successfully!")

 elif choice == "2":
 if students:
 for s in students:
 print(f"Name: {s['name']}, Age: {s['age']}")
 else:
 print("No students to display.")

 elif choice == "3":
 print("Goodbye!")
 break

 else:
 print("Invalid choice. Try again.")
```

Practice Tasks (Homework)

Apply your new dictionary skills to these exciting projects! Each one builds on the concepts you've learned today.

1

ATM Menu

Build a simple ATM interface that stores user accounts with balances. Implement features like check balance, deposit, and withdraw money using a dictionary to store account information.

2

Quiz Program

Create a quiz that stores questions and answers in a dictionary. Track user scores and provide feedback. Use nested dictionaries to organize different quiz categories.

3

Contact Storage App

Develop a contact book that stores names, phone numbers, and email addresses. Include features to add, view, and search for contacts using their names as keys.

Pro Tip

Start with a basic version of each project, then add features gradually. Use the student management code as a template for structure and user interaction!